

SymPy Bug Report

1 Bug 3: solveset returns nan as a zero of secant

1.1 Minimal Reproducer

```
from sympy import S, cos, sec, solveset, symbols

x = symbols("x")

solution = solveset(sec(x), x, domain=S.Complexes)
print("solveset(sec(x), x, domain=S.Complexes) =", solution)
print("sec(nan) =", sec(S.NaN))
print("solveset(1/cos(x), x, domain=S.Complexes) =",
      solveset(1 / cos(x), x, domain=S.Complexes))
```

1.2 Observed Behavior

```
SymPy version: 1.14.0
solveset(sec(x), x, domain=S.Complexes) = {nan}
sec(nan) = nan
solveset(1/cos(x), x, domain=S.Complexes) = EmptySet
```

SymPy returns a singleton set containing `nan` as a zero of $\sec(x)$. Direct evaluation of the reported candidate gives $\sec(\text{nan}) = \text{nan}$, not 0, and the mathematically equivalent reciprocal spelling $1/\cos(x)$ returns $\emptyset$.

1.3 Expected Behavior

The expected result is

$$\text{solveset}(\sec(x), x, \mathbb{C}) = \emptyset.$$

The value `nan` is not a complex number and should never be returned as a solution of the original equation.

1.4 Mathematical Explanation

For every finite complex number z , $\cos(z)$ is finite. Therefore

$$\sec(z) = \frac{1}{\cos(z)}$$

can never equal 0. If $\cos(z) \neq 0$, then $\sec(z)$ is a nonzero finite reciprocal. If $\cos(z) = 0$, then $\sec(z)$ has a pole and is undefined or infinite, not zero. Thus there is no finite complex z satisfying

$$\sec(z) = 0.$$

The returned value is not a harmless alternative representation or a branch choice: `nan` is not an element of the complex solution domain, and substitution does not satisfy the equation.

1.5 Source-Level Diagnosis

The diagnosis identifies the relevant source file as `sympy/solvers/solveset.py`. The public call `solveset(sec(x), x, domain=S.Complexes)` reaches `_solveset`; the fallback inversion path then calls `invert_complex(f, 0, symbol, domain)` around lines 2321–2324. `invert_complex` delegates to `_invert_complex`, which recognizes $\sec(x)$ as a trigonometric function around lines 577–578 and then calls `_invert_trig_hyp_complex`.

The problematic rule is in `_invert_trig_hyp_complex` around lines 462–466: the inversion helper treats $\sec$ like an ordinary invertible trigonometric function, chooses `asec` for the inverse, and constructs families of the form $2n\pi + \text{asec}(a)$ and $2n\pi - \text{asec}(a)$. For the equation $\sec(x) = 0$, this applies `asec(0)`. In SymPy 1.14.0, `asec(0)` evaluates to `zoo`; arithmetic such as $2n\pi \pm \text{zoo}$ simplifies during set construction to `{nan}`. The inversion code does not first enforce the reciprocal-trig range condition that $\sec$ and $\csc$ cannot take the value 0, and it does not later filter non-finite or `nan` candidates from the returned finite set.

```
# Simplified from the diagnosed inversion step:
# sec(x) = 0 is inverted as
x = 2*n*pi + asec(0)
x = 2*n*pi - asec(0)
```

This source-level behavior turns an impossible equation into invalid inverse images. A plausible fix direction supported by the diagnosis is to add range and singularity guards in the trigonometric inversion helper: for reciprocal trigonometric functions such as $\sec$ and $\csc$, exclude 0 from the target values before applying `asec` or `acsc`; if no target values remain, return `S.EmptySet`. A defensive cleanup could also discard `nan`, ∞ , $-\infty$, and complex infinity from finite solution sets.

1.6 Additional Failing Instances

The independent verification checks the representative case and five additional instances of the same form,

$$a \sec(bx + c),$$

where $a \neq 0$. Each expression has no complex zeros for the same reason: away from poles it is a nonzero scalar times a reciprocal of a finite cosine, and at poles it is undefined or infinite rather than zero. SymPy nevertheless returns `{nan}` for each listed case, while the equivalent reciprocal spelling $a/\cos(bx + c)$ returns $\emptyset$.

Input / Expression	SymPy behavior	Expected behavior
$\sec(x)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(x + 1)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(x - 2)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(2x)$	Returns <code>{nan}</code>	$\emptyset$
$\sec(3x + 1)$	Returns <code>{nan}</code>	$\emptyset$
$2\sec(x)$	Returns <code>{nan}</code>	$\emptyset$

1.7 Related Issues Assessment

The bug-finding harness performed a best-effort deduplication check and classified this as a new instance of a known failure mode. The closest related public material is SymPy's solveset documentation, SymPy's elementary-function documentation for reciprocal trigonometric functions, and the SymPy 1.14.0 source code showing the sec/csc inversion path through `asec/acsc`. The artifact did not identify a public issue, pull request, release note, Stack Overflow report, or discussion describing this exact return value for `solveset(sec(x), x, domain=S.Complexes)`. The general reciprocal-trig inversion failure mode may be broadly understood, but this exact reproducible case is previously unreported and remains individually actionable as an issue and regression test.

1.8 Proposed SymPy Regression Test

```
import pytest

from sympy import S, sec, solveset, symbols

@pytest.mark.parametrize(
    "scale, slope, offset",
    [
        (1, 1, 0),
        (1, 1, 1),
        (1, 1, -2),
        (1, 2, 0),
        (1, 3, 1),
        (2, 1, 0),
    ],
)
def test_solveset_sec_has_no_complex_zero(scale, slope, offset):
    x = symbols("x")
    assert solveset(scale * sec(slope * x + offset), x, domain=S.Complexes) == S.EmptySet
```